

Guia completo — CRUD de alunos com administrador, PostgreSQL e JWT

Material didático atualizado a partir do código existente no projeto em 4 de agosto de 2026.

1. Objetivo deste guia

Este documento explica o projeto do começo ao fim, com linguagem voltada para alunos que estão iniciando no desenvolvimento backend.

Ao terminar a leitura, o aluno deverá entender:

- o que é uma API;
- como o Express recebe requisições e envia respostas;
- como o projeto foi dividido em rotas, controllers e models;
- como o Node.js conversa com o PostgreSQL;
- como uma senha é protegida com bcrypt;
- como o administrador faz login;
- como um token JWT é criado;
- como o middleware valida esse token;
- como as rotas de alunos são protegidas;
- como funciona cada operação do CRUD;
- quais partes do código atual funcionam;
- qual erro ainda existe no código atual.

Este guia descreve o que realmente está nos arquivos. Ele não supõe que funcionalidades ainda não implementadas já existam.

2. Visão geral do sistema

O projeto possui duas entidades principais:

Administrador

É o usuário que realiza autenticação. Ele:

1. cadastra nome, e-mail e senha;
2. tem a senha transformada em hash;
3. faz login com e-mail e senha;
4. recebe um JWT;
5. envia o JWT para acessar rotas privadas.

Aluno

É o recurso administrado. O aluno não faz login. O administrador autenticado pode:

- cadastrar;

- listar;
- buscar pela matrícula;
- editar;
- excluir.

O fluxo geral é:

```
Administrador
  |
  | POST /admin/login
  v
Controller confere e-mail e senha
  |
  | dados corretos
  v
API gera um JWT
  |
  | Authorization: Bearer TOKEN
  v
Middleware valida o JWT
  |
  | token válido -> proximo()
  v
Controller de aluno
  |
  v
Model executa SQL no PostgreSQL
```

3. Tecnologias utilizadas

Tecnologia	Uso no projeto
Node.js	Executa JavaScript fora do navegador
Express 5	Cria servidor, rotas e middlewares
PostgreSQL	Armazena administradores e alunos
<code>pg</code>	Conecta o Node.js ao PostgreSQL
<code>dotenv</code>	Carrega as variáveis do arquivo <code>.env</code>
<code>bcryptjs</code>	Gera e compara hashes de senha
<code>jsonwebtoken</code>	Cria e valida tokens JWT

4. Conceitos fundamentais

4.1 O que é uma API?

Uma API é uma interface que permite a comunicação entre sistemas. Neste projeto, o cliente envia uma requisição HTTP e o servidor devolve JSON.

```
GET /aluno/listar
Authorization: Bearer TOKEN
```

Uma possível resposta é:

```
[
  {
    "matricula": 2026001,
    "nome": "Maria Silva",
    "email": "maria@email.com"
  }
]
```

4.2 O que é CRUD?

CRUD representa quatro operações básicas:

Operação	Significado	Método HTTP
Create	Criar	POST
Read	Consultar	GET
Update	Atualizar	PUT ou PATCH
Delete	Excluir	DELETE

4.3 O que é autenticação?

É o processo de confirmar a identidade de alguém. Aqui, isso acontece no login por meio do e-mail e da senha.

4.4 O que é autorização?

É o processo de decidir o que o usuário autenticado pode fazer.

O código atual possui autenticação JWT. Existe uma tentativa de autorização por perfil na rota de listagem, mas o método necessário ainda não foi implementado. Esse problema será explicado na seção de rotas de alunos.

4.5 O que é middleware?

Middleware é uma função executada entre a chegada da requisição e o controller.

Requisição -> Middleware -> Controller -> Model -> Banco

Se o token for inválido, o middleware responde e interrompe o fluxo. Se for válido, chama `proximo()`.

5. Estrutura atual do projeto

```
aluno/
├── docs/
├── src/
│   ├── config/
│   │   └── database.js
│   ├── middleware/
│   │   └── autenticacao.middleware.js
│   ├── modules/
│   │   ├── administrador/
│   │   │   ├── controllers/
│   │   │   │   └── administrador.controller.js
│   │   │   ├── models/
│   │   │   │   └── administrador.model.js
│   │   │   └── routes/
│   │   │       └── administrador.route.js
│   │   └── aluno/
│   │       ├── controllers/
│   │       │   └── aluno.controller.js
│   │       ├── models/
│   │       │   └── aluno.model.js
│   │       └── routes/
│   │           └── aluno.route.js
│   ├── utils/
│   │   └── utils.js
│   └── index.js
├── .env
├── .env.example
├── package.json
└── README.md
```

Responsabilidade de cada camada

Camada	Pergunta que responde
Route	Qual endereço chama qual função?
Middleware	A requisição pode continuar?
Controller	Como validar a entrada e montar a resposta HTTP?
Model	Qual consulta SQL deve ser executada?
Database	Como abrir conexão com o PostgreSQL?

6. Preparação do projeto

6.1 Instalar dependências

Na raiz do projeto:

```
npm install
```

6.2 Entendendo o `package.json`

O arquivo atual é:

```
{
  "name": "aluno",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "start": "node --watch src/index.js"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "pg": "^8.22.0"
  }
}
```

Explicação:

- `name`: nome do pacote;
- `version`: versão do projeto;
- `type: module`: permite usar `import` e `export`;
- `scripts.start`: inicia `src/index.js`;
- `--watch`: reinicia o Node quando um arquivo é alterado;
- `dependencies`: bibliotecas necessárias em execução.

Para iniciar:

```
npm start
```

7. Variáveis de ambiente

O `.env.example` mostra as variáveis necessárias:

```
PORTA=adicionar aqui uma porta
PGUSER=
PGPASSWORD=
PGHOST=
PGPORT=
PGDATABASE=
JWT_SECRET=
JWT_TEMPO_EXPIRACAO=1h
```

Exemplo local:

```
PORTA=3000
PGUSER=postgres
PGPASSWORD=sua_senha
PGHOST=localhost
PGPORT=5432
PGDATABASE=escola
JWT_SECRET=uma_chave_longa_e_dificil_de_adivinhar
JWT_TEMPO_EXPIRACAO=1h
```

Variável	Função
PORTA	Porta do servidor Express
PGUSER	Usuário do PostgreSQL
PGPASSWORD	Senha do PostgreSQL
PGHOST	Endereço do servidor PostgreSQL
PGPORT	Porta do PostgreSQL
PGDATABASE	Nome do banco
JWT_SECRET	Segredo que assina e valida o JWT
JWT_TEMPO_EXPIRACAO	Validade do token

O `.env` não deve ser versionado. O `.gitignore` já contém regras para ignorá-lo.

8. Banco de dados

O código espera estas tabelas:

```
CREATE TABLE admins (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,
```

```
email VARCHAR(150) NOT NULL UNIQUE,  
senha VARCHAR(255) NOT NULL,  
ativo BOOLEAN NOT NULL DEFAULT TRUE,  
criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE aluno (  
  matricula INTEGER PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE  
);
```

Tabela `admins`

- `id`: identificador gerado automaticamente;
- `nome`: nome do administrador;
- `email`: deve ser único;
- `senha`: guarda o hash bcrypt, nunca a senha pura;
- `ativo`: controla se o administrador pode fazer login;
- `criado_em`: registra a data de criação.

Tabela `aluno`

- `matricula`: identificador numérico;
- `nome`: nome do aluno;
- `email`: e-mail único.

No banco atual, matrícula é inteira. Exemplos como `a92222` presentes em alguns comentários antigos não combinam com esse tipo.

9. Arquivo `src/config/database.js`

Código atual

```
import pg from "pg";  
import dotenv from "dotenv";  
  
dotenv.config();  
  
const { Pool } = pg;  
  
const conexao = new Pool({  
  user: process.env.PGUSER,  
  password: process.env.PGPASSWORD,  
  host: process.env.PGHOST,  
  port: Number(process.env.PGPORT ?? 5432),  
  database: process.env.PGDATABASE,  
});
```

```
try {
  await conexao.query(`select 1 + 1`);
  console.log("Conexao realizada com sucesso!");
} catch (error) {
  console.error({ mensagem: "Erro ao iniciar banco", erro: error.message });
}

export default conexao;
```

Explicação linha por linha

```
import pg from "pg";
```

Importa o pacote que permite ao Node conversar com PostgreSQL.

```
import dotenv from "dotenv";
```

Importa a biblioteca que lê o `.env`.

```
dotenv.config();
```

Carrega os valores do `.env` para `process.env`.

```
const { Pool } = pg;
```

Retira a classe `Pool` do objeto `pg`. Uma pool reutiliza conexões em vez de abrir uma nova para cada consulta.

```
const conexao = new Pool({...});
```

Cria a pool usando as variáveis do ambiente.

```
Number(process.env.PGPORT ?? 5432)
```

- lê `PGPORT`;
- se estiver ausente, usa `5432`;
- converte o texto para número.

```
await conexao.query('select 1 + 1');
```

Executa uma consulta simples para testar se o banco responde.

```
export default conexao;
```

Exporta a pool para os modelos.

10. Arquivo principal `src/index.js`

Código atual

```
import express from "express";
import dotenv from "dotenv";
```



```
import router from "./modules/aluno/routes/aluno.route.js";
import routerAdmin from "./modules/administrador/routes/administrador.route.js";

dotenv.config();

const app = express();

app.use(express.json());

app.use("/aluno", router);
app.use("/admin", routerAdmin);

const porta = process.env.PORTA;

app.get("/", (requisicao, resposta) => {
  try {
    resposta.status(200).json({
      mensagem: "API funcionando com sucesso!",
      status: "ok",
      date: new Date().toLocaleString("pt-BR", {
        timeZone: "America/Recife"
      })
    });
  } catch (error) {
    resposta.status(500).json({
      mensagem: "Erro ao iniciar API!",
      erro: error.message
    });
  }
});

app.listen(porta, () => {
  console.log(`O servidor esta em execucao na porta ${porta}!`);
});
```

Explicação

```
import express from "express";
```

Importa o framework Express.

Os imports de `router` e `routerAdmin` carregam as rotas dos dois módulos.

```
const app = express();
```

Cria a aplicação.

```
app.use(express.json());
```

Converte o body JSON em objeto JavaScript. Sem isso, `requisicao.body` pode ficar indefinido.

```
app.use("/aluno", router);
```

Adiciona `/aluno` antes das rotas do módulo. Uma rota interna `/listar` vira `/aluno/listar`.

```
app.use("/admin", routerAdmin);
```

Adiciona `/admin` antes das rotas administrativas.

```
app.get("/", ...)
```

Cria uma rota pública para verificar se a API está no ar.

```
app.listen(porta, ...)
```

Inicia o servidor na porta definida no `.env`.

11. Model do administrador

Arquivo: `src/modules/administrador/models/administrador.model.js`.

11.1 Importação e classe

```
import conexao from "../../../config/database.js";

class AdministradorModel {
```

O import recebe a pool. A classe agrupa consultas relacionadas ao administrador.

11.2 Cadastrar

```
static async cadastrar(nome, email, hashSenha) {
  const dados = [nome, email, hashSenha];
  const query = `
    insert into admins(nome, email, senha)
    values ($1, $2, $3)
    returning id, nome, email, ativo, criado_em
  `;
  const resultado = await conexao.query(query, dados);
  return resultado.rows[0];
}
```

Linha a linha:

- `static`: permite chamar `AdministradorModel.cadastrar()` sem `new`;
- `async`: permite usar `await`;
- recebe `hashSenha`, não a senha pura;
- `dados` guarda valores na mesma ordem de `$1`, `$2` e `$3`;
- `INSERT` cria o registro;
- `RETURNING` devolve somente campos seguros;
- a senha não aparece no retorno;

- `resultado.rows[0]` devolve o primeiro registro criado.

11.3 Contar todos os administradores

```
static async contarAdmins() {  
  const query = `select count(*) from admins`;   
  const resultado = await conexao.query(query);  
  return Number(resultado.rows[0].count);  
}
```

O PostgreSQL devolve `COUNT(*)` como texto. `Number(...)` converte para número.

Observação: esse método existe, mas o cadastro atual usa `verificaAdminsAtivos()`.

11.4 Contar administradores ativos

```
static async verificaAdminsAtivos() {  
  const query = `select count(*) from admins  
  where ativo = true`;   
  const resultado = await conexao.query(query);  
  return Number(resultado.rows[0].count);  
}
```

Essa consulta conta apenas registros com `ativo = true`. Assim, o cadastro bloqueia um novo administrador quando já existe outro ativo.

Isso significa que o sistema atual pode ter vários registros históricos, mas pretende permitir somente um ativo por vez.

11.5 Buscar por e-mail

```
static async buscarPorEmail(email) {  
  const dados = [email];  
  const query = `   
    select id, nome, email, senha, ativo, criado_em from admins  
    where email = $1  
  `;  
  const resultado = await conexao.query(query, dados);  
  return resultado.rows[0];  
}
```

Essa consulta é usada no login. Ela precisa trazer `senha`, pois o `bcrypt` compara o hash.

Se não houver resultado, `resultado.rows[0]` será `undefined`.

11.6 Buscar perfil por ID

```
static async buscarPerfilPorId(id) {
  const dados = [id];
  const query = `
    select id, nome, email from admins
    where id = $1
  `;
  const resultado = await conexao.query(query, dados);
  return resultado.rows[0];
}
```

Esta consulta não seleciona a senha. Ela devolve somente `id`, `nome` e `email`.

12. Controller do administrador

Arquivo: `src/modules/administrador/controllers/administrador.controller.js`.

12.1 Imports

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import AdministradorModel from "../models/administrador.model.js";
```

- `bcrypt`: hash e comparação de senha;
- `jwt`: criação do token;
- `AdministradorModel`: acesso ao banco.

12.2 Cadastro

```
const { nome, email, senha } = requisicao.body;
```

Desestrutura os três campos enviados em JSON.

```
if (!nome || !email || !senha) {
  return resposta.status(400).json({
    mensagem: "Todos os campos sao obrigatorios!"
  });
}
```

Se algum campo for vazio ou ausente, responde `400`. O `return` impede a continuação.

```
const totalAdmin = await AdministradorModel.verificaAdminsAtivos();
if (totalAdmin > 0) {
```

```
return resposta.status(409).json({
  mensagem: "Existe um administrador cadastrado e ativo no sistema!"
});
}
```

Consulta quantos administradores ativos existem. 409 `Conflict` informa conflito com o estado atual.

```
if (senha.length < 8) {
```

Recusa senha com menos de oito caracteres.

```
const regexSenha = /^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[!@#$%^&*()_+\-=\[\]
{};':"\\|,.<>/?]).{8,32}$/;
```

A expressão exige:

- `(?=.*[A-Z])`: uma maiúscula;
- `(?=.*[a-z])`: uma minúscula;
- `(?=.*[0-9])`: um número;
- o próximo grupo: um caractere especial;
- `.{8,32}`: entre 8 e 32 caracteres.

```
const regexEmail = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9-]+(?:\.[a-zA-Z0-9-]+)+$/;
```

Valida a estrutura básica do e-mail.

O projeto usa nome e e-mail exatamente como recebidos. Não existe `trim()` nem `toLowerCase()`.

```
const hashSenha = await bcrypt.hash(senha, 10);
```

Gera o hash. O valor 10 é o custo do bcrypt.

```
const administrador = await AdministradorModel.cadastrar(
  nome,
  email,
  hashSenha
);
```

Envia o hash ao model. O model chama esse terceiro parâmetro de `hashSenha`; a associação ocorre pela posição.

12.3 Login

O login recebe:

```
{
  "email": "admin@email.com",
  "senha": "Senha@123"
}
```

```
const administrador = await AdministradorModel.buscarPorEmail(email);
```

Procura o registro. Como não há normalização, o e-mail deve ser enviado da mesma forma usada no cadastro.

```
if (!administrador) {
  return resposta.status(401).json({
    mensagem: "E-mail ou senha incorretos!"
  });
}
```

Evita acessar propriedades de um registro inexistente e usa mensagem genérica.

```
if (administrador.ativo === false) {
```

Impede login de administrador inativo.

```
const senhaCorreta = await bcrypt.compare(senha, administrador.senha);
```

Compara a senha recebida com o hash armazenado. A senha não é descriptografada.

```
const token = jwt.sign(
  {
    id: administrador.id,
    nome: administrador.nome,
    email: administrador.email
  },
  process.env.JWT_SECRET,
  {
    expiresIn: process.env.JWT_TEMPO_EXPIRACAO
  }
);
```

`jwt.sign` recebe:

1. payload com `id`, `nome` e `email`;
2. segredo do `.env`;
3. configuração de expiração.

Senha nunca deve ser colocada no payload, pois o conteúdo do JWT pode ser lido.

12.4 Perfil

```
const idDoToken = requisicao.usuario.id;
```

O middleware criou `requisicao.usuario`. O controller aproveita o `id` validado do token.

```
const administrador = await AdministradorModel.buscarPerfilPorId(idDoToken);
```

Busca o perfil pelo ID e não pelo valor fornecido livremente na URL.

Se o registro não existir, responde `404`. Caso exista, devolve o administrador sem senha.

13. JWT explicado do zero

JWT significa JSON Web Token. Um token costuma ter três partes:

```
HEADER.PAYLOAD.SIGNATURE
```

Header

Informa o tipo do token e o algoritmo.

Payload

Contém os dados passados ao `jwt.sign`, além de datas como:

- `iat`: emissão;
- `exp`: expiração.

Signature

Permite verificar se o conteúdo foi alterado e se foi assinado pelo servidor.

JWT não criptografa automaticamente o payload. Não coloque senha, segredo ou dado sensível nele.

14. Middleware de autenticação

Arquivo: `src/middleware/autenticacao.middleware.js`.

Código atual

```
import jwt from "jsonwebtoken";

class AutenticacaoMiddleware {
  static autenticar(requisicao, resposta, proximo) {
    const autorizacao = requisicao.headers.authorization;

    if (!autorizacao) {
      return resposta.status(401).json({
        mensagem: "Token de autenticacao nao fornecido!"
      });
    }

    const token = autorizacao.split(" ")[1];

    if (!token) {
      return resposta.status(401).json({
        mensagem: "Token mal formatado. Use: Bearer TOKEN"
      });
    }

    jwt.verify(token, process.env.JWT_SECRET, (erro, usuario) => {
      if (erro) {
        return resposta.status(403).json({
          mensagem: "Acesso nao autorizado!"
        });
      }

      requisicao.usuario = usuario;

      return proximo();
    });
  }
}

export default AutenticacaoMiddleware;
```

Explicação linha por linha

```
import jwt from "jsonwebtoken";
```

Importa a biblioteca usada para validar tokens.

```
static autenticar(requisicao, resposta, proximo)
```

Recebe três objetos/funções do Express:

- `requisicao`: dados recebidos;

- **resposta**: forma de responder;
- **proximo**: libera a próxima etapa.

```
const autorizacao = requisicao.headers.authorization;
```

Lê o cabeçalho:

```
Authorization: Bearer TOKEN
```

```
if (!autorizacao)
```

Se o cabeçalho não foi enviado, responde **401**.

```
const token = autorizacao.split(" ")[1];
```

Divide a string no espaço:

```
Bearer abc.def.ghi
  |      |
  [0]    [1]
```

A posição **[1]** contém o token.

```
if (!token)
```

Detecta um cabeçalho sem a segunda parte.

```
jwt.verify(token, process.env.JWT_SECRET, callback)
```

Valida assinatura e expiração.

```
(erro, usuario) => {
```

O callback recebe:

- **erro**: token inválido ou expirado;
- **usuario**: payload decodificado quando válido.

O segundo parâmetro não é o **proximo** do Express.

```
requisicao.usuario = usuario;
```

Acrescenta o payload validado à requisição. Por isso o controller consegue usar **requisicao.usuario.id**.

```
return proximo();
```

Libera o próximo middleware ou controller da rota.

15. Rotas do administrador

Arquivo: `src/modules/administrador/routes/administrador.route.js`.

```
import express from "express";
import AdministradorController from "../controllers/administrador.controller.js";
import AutenticacaoMiddleware from
"../../middleware/autenticacao.middleware.js";

const routerAdmin = express.Router();

routerAdmin.post("/cadastrar", AdministradorController.cadastrar);
routerAdmin.post("/login", AdministradorController.login);

routerAdmin.get(
  "/perfil",
  AutenticacaoMiddleware.autenticar,
  AdministradorController.perfil
);

export default routerAdmin;
```

Rotas públicas

Cadastro e login não exigem token:

```
POST /admin/cadastrar
POST /admin/login
```

Rota privada

```
GET /admin/perfil
```

Ordem de execução:

1. `AutenticacaoMiddleware.autenticar`;
2. se válido, `proximo()`;
3. `AdministradorController.perfil`.

16. Model de aluno

Arquivo: `src/modules/aluno/models/aluno.model.js`.

Todos os métodos são `static async`, pois executam SQL assíncrono.

16.1 Cadastrar

```
const dados = [matricula, nome, email];
const query = `
  insert into aluno (matricula, nome, email)
  values ($1, $2, $3)
  returning *
`;
const resultado = await conexao.query(query, dados);
return resultado.rows[0];
```

- `$1`, `$2` e `$3` são parâmetros SQL;
- valores ficam separados do texto SQL;
- `returning *` devolve o aluno criado;
- `rows[0]` devolve o primeiro registro.

16.2 Listar todos

```
const resultado = await conexao.query(`select * from aluno`);
return resultado.rows;
```

Aqui retorna o array completo, não somente `[0]`.

16.3 Buscar pela matrícula

```
const dados = [matricula];
const query = `select * from aluno where matricula = $1`;
```

Se não encontrar, `rows[0]` será `undefined`.

16.4 Editar totalmente

Primeiro busca o aluno. Se não existir, retorna `null`.

```
UPDATE aluno
SET nome = $2, email = $3
WHERE matricula = $1
RETURNING *
```

PUT substitui os dois campos editáveis.

16.5 Editar parcialmente

```
UPDATE aluno
SET nome = COALESCE($2, nome),
    email = COALESCE($3, email)
WHERE matricula = $1
RETURNING *
```

COALESCE escolhe o primeiro valor que não seja **NULL**. Se **\$2** não foi informado, mantém **nome**.

16.6 Excluir pela matrícula

```
DELETE FROM aluno
WHERE matricula = $1
RETURNING *
```

O registro excluído é devolvido ao controller.

16.7 Excluir todos

```
DELETE FROM aluno RETURNING *
```

Remove todos e retorna um array com os registros removidos.

17. Controller de aluno

Arquivo: `src/modules/aluno/controllers/aluno.controller.js`.

17.1 Cadastrar

```
const { matricula, nome, email } = requisicao.body;
```

Lê o body.

```
if (!matricula || !nome || !email)
```

Exige os três campos.

```
const alunoCadastrado = await AlunoModel.cadastrar(...);
```

Espera o banco finalizar.

Responde **201 Created** com mensagem e aluno.

17.2 Listar todos

Chama `AlunoModel.listarTodos()`. Se o array tiver tamanho zero, responde **200** com uma mensagem. Caso contrário, devolve o array.

17.3 Buscar pela matrícula

```
const { matricula } = requisicao.params;
```

Em `/listar/2026001`, o valor vem de `params`.

Se o model devolver `undefined`, responde **404**.

17.4 PUT

Exige `nome` e `email`. Se o aluno existir, atualiza ambos.

17.5 PATCH

```
if (!nome && !email)
```

Exige pelo menos um campo. O model usa `COALESCE` para manter o outro.

17.6 Excluir todos

Chama o model e responde com mensagem e array `alunos` contendo os registros excluídos.

17.7 Excluir pela matrícula

Se não encontrar, responde **404**; se encontrar, devolve o aluno removido.

17.8 try/catch

Cada método usa `try/catch`. Erros de banco são transformados em resposta **500**.

18. Rotas de alunos e erro atual

Arquivo: `src/modules/aluno/routes/aluno.route.js`.

Quase todas as rotas seguem este formato:

```
router.post(  
  "/cadastrar",  
  AutenticacaoMiddleware.autenticar,
```

```
AlunoController.cadastrar  
);
```

Isso significa:

1. recebe **POST** `/aluno/cadastrar`;
2. autentica o JWT;
3. chama o controller.

Atenção: linha problemática no código atual

A rota de listar todos está assim:

```
router.get(  
  "/listar",  
  AutenticacaoMiddleware.autenticar,  
  AutenticacaoMiddleware.autorizar['professor', 'pedagogico'],  
  AlunoController.listarTodos  
);
```

Há dois problemas:

1. `AutenticacaoMiddleware.autorizar` não existe na classe atual;
2. `['professor', 'pedagogico']` não chama uma função de autorização; em JavaScript, essa expressão com vírgula resulta na chave `'pedagogico'`.

Ao carregar o arquivo, tentar acessar uma propriedade de `undefined` pode gerar erro e impedir a API de iniciar.

Uma autorização por perfil exigiria primeiro implementar um método próprio. Exemplo conceitual, ainda não existente no projeto:

```
static autorizar(perfisPermitidos) {  
  return (requisicao, resposta, proximo) => {  
    if (!perfisPermitidos.includes(requisicao.usuario.perfil)) {  
      return resposta.status(403).json({  
        mensagem: "Usuario sem permissao!"  
      });  
    }  
  
    return proximo();  
  };  
}
```

E a rota seria conceitualmente:

```
AutenticacaoMiddleware.autorizar(["professor", "pedagogico"])
```

Porém, o banco e o payload JWT atuais não possuem campo `perfil`. Portanto, adicionar somente esse método não seria suficiente.

Para refletir apenas a autenticação já implementada, a rota teria de usar somente `AutenticacaoMiddleware.autenticar`. Este guia não altera a aplicação; apenas aponta o estado atual.

19. Tabela de rotas pretendidas pelo código

Públicas

Método	Rota	Função
GET	/	Teste da API
POST	/admin/cadastrar	Cadastro do administrador
POST	/admin/login	Login e geração do JWT

Privadas

Método	Rota	Função
GET	/admin/perfil	Perfil do token
GET	/aluno/listar	Lista todos; atualmente contém erro em <code>autorizar</code>
GET	/aluno/listar/:matricula	Busca aluno
POST	/aluno/cadastrar	Cadastra aluno
PUT	/aluno/editar/total/:matricula	Atualiza tudo
PATCH	/aluno/editar/parcial/:matricula	Atualiza parcialmente
DELETE	/aluno/excluir/todos	Exclui todos
DELETE	/aluno/excluir/:matricula	Exclui um

20. Status HTTP usados

Código	Nome	Uso no projeto
200	OK	Login, consulta, edição e exclusão
201	Created	Cadastro concluído
400	Bad Request	Campos ausentes ou inválidos
401	Unauthorized	Credenciais incorretas ou token ausente

Código	Nome	Uso no projeto
403	Forbidden	Admin inativo ou token rejeitado
404	Not Found	Registro não encontrado
409	Conflict	Já existe administrador ativo
500	Internal Server Error	Erro inesperado ou de banco

21. Testes manuais com PowerShell

Antes de testar, corrija ou remova a referência inexistente a `autorizar` da rota `/aluno/listar`, pois ela pode impedir o servidor de iniciar.

21.1 Iniciar

```
npm start
```

21.2 Testar a raiz

```
Invoke-RestMethod -Method Get -Uri "http://localhost:3000/"
```

21.3 Cadastrar administrador

```
$corpoAdmin = @{  
    nome = "Administrador"  
    email = "admin@email.com"  
    senha = "Senha@123"  
} | ConvertTo-Json  
  
Invoke-RestMethod `   
    -Method Post `   
    -Uri "http://localhost:3000/admin/cadastrar" `   
    -ContentType "application/json" `   
    -Body $corpoAdmin
```

21.4 Fazer login

```
$corpoLogin = @{  
    email = "admin@email.com"  
    senha = "Senha@123"  
} | ConvertTo-Json  
  
$login = Invoke-RestMethod `
```



```
-Method Post `
-Uri "http://localhost:3000/admin/login" `
-ContentType "application/json" `
-Body $corpoLogin
```

21.5 Guardar token e header

```
$token = $login.token
$cabecalhos = @{ Authorization = "Bearer $token" }
```

21.6 Consultar perfil

```
Invoke-RestMethod `
-Method Get `
-Uri "http://localhost:3000/admin/perfil" `
-Headers $cabecalhos
```

21.7 Cadastrar aluno

```
$corpoAluno = @{
    matricula = 2026001
    nome = "Maria Silva"
    email = "maria@email.com"
} | ConvertTo-Json

Invoke-RestMethod `
-Method Post `
-Uri "http://localhost:3000/aluno/cadastrar" `
-Headers $cabecalhos `
-ContentType "application/json" `
-Body $corpoAluno
```

21.8 Buscar aluno

```
Invoke-RestMethod `
-Method Get `
-Uri "http://localhost:3000/aluno/listar/2026001" `
-Headers $cabecalhos
```

21.9 Editar parcialmente

```
$alteracao = @{ nome = "Maria Oliveira" } | ConvertTo-Json

Invoke-RestMethod `
  -Method Patch `
  -Uri "http://localhost:3000/aluno/editar/parcial/2026001" `
  -Headers $cabecalhos `
  -ContentType "application/json" `
  -Body $alteracao
```

21.10 Excluir aluno

```
Invoke-RestMethod `
  -Method Delete `
  -Uri "http://localhost:3000/aluno/excluir/2026001" `
  -Headers $cabecalhos
```

22. Como testar em Postman, Insomnia ou Thunder Client

Rotas públicas

1. escolha o método;
2. informe a URL;
3. para POST, selecione Body, JSON;
4. envie o objeto.

Rotas privadas

1. faça login;
2. copie o token;
3. abra Authorization;
4. escolha Bearer Token;
5. cole apenas o token;
6. envie a requisição.

O header final deve ser:

```
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

23. Conceitos JavaScript importantes

async e await

async declara que uma função trabalha de forma assíncrona. **await** espera uma Promise terminar.

```
const administrador = await AdministradorModel.buscarPorEmail(email);
```

Sem `await`, a variável receberia uma Promise pendente.

Desestruturação

```
const { email, senha } = requisicao.body;
```

Cria variáveis a partir de propriedades do objeto.

return

```
return resposta.status(400).json({ mensagem: "Erro" });
```

Envia a resposta e encerra a função, evitando uma segunda resposta.

try/catch

```
try {  
  // operação  
} catch (error) {  
  // tratamento  
}
```

Captura falhas assíncronas aguardadas com `await`.

Array `rows`

O `pg` devolve resultados em `resultado.rows`.

- `rows`: todos os registros;
- `rows[0]`: primeiro registro;
- `rows.length`: quantidade retornada.

24. Segurança presente

- hash bcrypt em vez de senha pura;
- consultas parametrizadas com `$1`, `$2`, `$3`;
- JWT assinado com segredo;
- expiração configurável;
- middleware nas rotas privadas;
- senha omitida no cadastro e perfil;

- mensagem genérica no login incorreto;
 - verificação de administrador ativo.
-

25. Limitações e pontos de atenção atuais

1. A rota `/aluno/listar` referencia um método `autorizar` inexistente.
2. Não existe coluna `perfil` no esquema apresentado nem no JWT atual.
3. Não existe refresh token.
4. Não existe logout com revogação de JWT.
5. Não existe recuperação de senha.
6. Não existe rate limiting no login.
7. Não existem testes automatizados.
8. `error.message` é devolvido ao cliente em respostas `500`.
9. Nome e e-mail não são normalizados.
10. Matrícula é numérica no banco atual.
11. `contarAdmins()` existe, mas o cadastro usa `verificaAdminsAtivos()`.
12. Alguns comentários do model de aluno ainda dizem "array", embora os dados estejam no PostgreSQL.

Esses itens são oportunidades de evolução, não funcionalidades já prontas.

26. Erros comuns

API não inicia e menciona `autorizar`

Causa: a rota tenta acessar `AutenticacaoMiddleware.autorizar`, que não existe.

Token de autenticacao nao fornecido!

O header `Authorization` não foi enviado.

Token mal formatado

O valor não possui uma segunda parte depois do espaço.

Acesso nao autorizado!

O token é inválido, expirou ou foi assinado com outro segredo.

Login falha após cadastro

Confira se e-mail e senha foram enviados exatamente como cadastrados. O projeto não normaliza e-mail.

Erro de conexão

Confira as variáveis `PGUSER`, `PGPASSWORD`, `PGHOST`, `PGPORT` e `PGDATABASE`.

Violação de chave única

O banco não aceita e-mails duplicados nem matrículas repetidas.

27. Fluxo completo para revisão

CADASTRO DO ADMINISTRADOR

Body -> validações -> conta admins ativos -> bcrypt.hash
-> INSERT no PostgreSQL -> resposta sem senha

LOGIN

Body -> busca por e-mail -> verifica ativo -> bcrypt.compare
-> jwt.sign -> devolve token

ROTA PRIVADA

Authorization -> extrai token -> jwt.verify
-> requisicao.usuario -> proximo()
-> controller -> model -> PostgreSQL

CRUD DE ALUNO

Rota -> autenticação -> controller valida entrada
-> model executa SQL parametrizado -> resposta JSON

28. Resumo final

O projeto ensina uma API em camadas com persistência real e autenticação JWT.

Os pontos centrais são:

- o administrador é quem se autentica;
- o bcrypt protege a senha;
- o login gera o JWT;
- o cliente envia **Authorization: Bearer TOKEN**;
- o middleware valida e cria **requisicao.usuario**;
- os controllers tratam HTTP;
- os models tratam SQL;
- o PostgreSQL persiste os dados;
- o CRUD de alunos deve ficar atrás da autenticação.

Antes de executar a versão atual, é necessário observar o erro da referência a

AutenticacaoMiddleware.autorizar na rota **/aluno/listar**. Todo o restante deste documento foi escrito para representar fielmente o código atual, sem apresentar como pronta uma autorização por perfil que ainda não foi implementada.